

Decisión Recomendada

Plano interactivo 2D — Primera iteración del MVP

- Android / Jetpack Compose
- Kotlin
- MVP
- Sin Room

RENDERIZADO	GESTOS	PERSISTENCIA
Compose Canvas	pointerInput + detectTransformGestures	Solo API (Retrofit → .NET)

1. POR QUÉ COMPOSE CANVAS

El plano se construye desde una lista de zonas que provienen del backend — no es un SVG pre-dibujado. Esto descarta las opciones basadas en cargar archivos SVG (Coil, AndroidSVG) porque no aportan nada: la app igualmente tendría que dibujar las zonas encima.

Compose Canvas es la opción nativa, soportada oficialmente por Google, con integración directa al estado de Compose (State, Flow, ViewModel). Sin overhead de interoperar con Android Views.

2. POR QUÉ EMPEZAR SIN ROOM

Room es útil, pero **solo si la app realmente necesita datos cacheados localmente**. Agregarla «por si acaso» trae costos concretos:

- **El backend ya es la fuente de verdad.** Duplicar datos en local introduce sincronización: ¿qué pasa si la copia local difiere del servidor? Cada pregunta requiere código adicional y es fuente de bugs.
- **Arquitectura más simple = MVP más rápido.** Solo API: Retrofit → ViewModel → Compose (3 capas). Con Room: Retrofit + Room → Repository con sync → ViewModel → Compose. Más capas, más superficie de error.
- **No hay requisitos confirmados de modo offline.** Si producto no ha pedido que la app funcione sin conexión, asumir que sí lo necesitará es sobre-ingeniería.
- **Migrar después es barato.** Si la UI consume un Repository (no Retrofit directamente), agregar Room más adelante es solo cambiar la implementación del Repository — sin tocar ViewModel ni UI.

3. CUÁNDO SÍ MIGRAR A ROOM (API + ROOM)

TRIG GER	ESCENARIO
■	Producto confirma que la app debe funcionar sin conexión — p.ej. técnicos de campo sin señal.
■	La latencia del endpoint genera mala UX (esperas de 2-3 s al abrir el plano).
—■	Se necesita edición muy fluida donde el round-trip al servidor introduce lag perceptible (optimistic updates son mucho más fáciles con Room).

Si ocurre alguno de estos triggers, se abre una issue para agregar Room encima del repositorio existente. Es una refactorización acotada, no una reescritura.

4. STACK MÍNIMO DEL MVP

Capa	Librería / API	Propósito
UI	Jetpack Compose	Composables del plano y de la app
Renderizado	Compose Canvas + pointerInput	Dibujar zonas y manejar gestos
Estado de UI	ViewModel + StateFlow	Coordinar lógica y exponer estado a la UI
Red	Retrofit + OkHttp + Kotlinx Serialization	Consumir el endpoint .NET
Inyección de dep.	Hilt (o manual)	Conectar las capas

5. RECORDATORIO IMPORTANTE

Esta es una recomendación técnica, no una decisión final del equipo.

- La elección entre «solo API» y «API + Room» depende de los requisitos reales del producto.
- Solo el equipo (con producto y diseño) puede confirmar esos requisitos.
- El spike aporta: contexto técnico, consecuencias de cada opción y un punto de partida razonable.